



Hochschule München | University of Applied Sciences

Pflichtenheft

Sensorbasiertes Verschleißtracking für LKW-Flotten

Predictive Maintenance – Big Data Projekt

Vorlesung Big Data and Large Scale Systeme

Dozent: Dr. Oliver Weisshuhn

Gruppe: Team 1

Ujjwal Makkar

Edel Cako

Maximilian Plitzko

Walter Seitz

1. Einführung	3
Projektumfang (Scope)	3
Abgrenzung (Nicht im Scope)	3
2. Personas und deren Bedürfnisse	4
3. Value Proposition Canvas	6
4. Anforderungen	7
4.1 Funktionale Anforderungen	7
4.2 Nichtfunktionale Anforderungen	10
5. Anwendungsfall	4
5.1 Aktivitätsdiagramm	12
5.2 UI-Wireframes	13
Screen 1: Flottenübersicht	13
Screen 2: Einzelfahrzeug-Detail	13
Screen 3: Alert-Feed	14
6. Architektur	15
6.1 Systemarchitektur – 5-Schichten-Modell	15
6.2 Datenfluss	15
6.3 Technologie-Stack	17
6.4 Datenquellen	18
7. Projektabgrenzung und Ausblick	19
7.1 Was das System bewusst nicht leisten wird (MVP)	19
7.2 Zielbild nach dem MVP	19

1. Einführung

LKW-Flotten sind das logistische Rückgrat Deutschlands – und gleichzeitig einer der kostenintensivsten Betriebsbereiche der Speditionsbranche. Ungeplante Fahrzeugausfälle verursachen durchschnittlich 500–700 € Kosten pro Stillstandsstunde, zuzüglich Vertragsstrafen und Reputationsschäden durch Lieferverzögerungen.

Das vorliegende Dokument beschreibt die Anforderungen an ein sensorbasiertes Predictive-Maintenance-System für LKW-Flotten. Das System erfasst (simulierte) OBD-II-Telemetriedaten in Echtzeit, verarbeitet diese durch eine Machine-Learning-Pipeline und stellt Flottenmanagern, Fahrern und Werkstattleitern automatisierte Verschleißwarnungen sowie Restlebensdauerprognosen bereit.

Da für dieses Hochschulprojekt keine physische Hardware zur Verfügung steht, wird die gesamte Sensorik durch einen Python-basierten Simulator ersetzt. Der KIT/Kaggle Datensatz (Automotive Engine Health) bildet die Trainingsgrundlage für die ML-Modelle.

Projektumfang (Scope)

Das System richtet sich an drei Nutzergruppen, die über ein SaaS-Abonnementmodell bedient werden:

- Flottenmanager (Entscheider) – Web-Dashboard und E-Mail-Alerts
- LKW-Fahrer (Endnutzer) – Push-Benachrichtigungen bei kritischen Zuständen
- Werkstattleiter (Umsetzer) – Wartungskalender-Integration und Teilebestellübersicht

Abgrenzung (Nicht im Scope)

Folgende Aspekte sind im Rahmen dieses Projekts explizit ausgeschlossen:

- Einsatz echter Hardware (kein OBD-II-Dongle / ELM327 im MVP)
- Produktiver Kafka- oder MQTT-Broker (Datentransport erfolgt per Batch-CSV)
- ERP- oder SAP-Integration
- Native Mobile App (iOS/Android)
- LSTM/Deep-Learning-Modelle im MVP (Zielbild nach MVP)
- Vollständige DSGVO-Compliance-Prüfung (nur simulierte Pseudodaten)

2. Personas und deren Bedürfnisse

Um die Systemanforderungen zielgruppengerecht abzuleiten, werden drei repräsentative Nutzerprofile (Personas) definiert. Sie decken alle relevanten Rollen im täglichen Flottenbetrieb ab und bilden die Grundlage für die nachfolgende Value-Proposition-Analyse sowie die funktionalen Anforderungen.

Persona 1: Thomas Müller – Flottenmanager (Entscheider)	
Alter: 48 Jahre Arbeitgeber: Spedition Müller GmbH Verantwortlich für: 120 LKW im Fernverkehr	
Ziele	Ungeplante Fahrzeugausfälle minimieren und Wartungskosten planbar halten. Return-on-Investment der Flottenlösung gegenüber Geschäftsführung nachweisen.
Herausforderungen	- Kein Echtzeit-Überblick über den Komponentenzustand der Flotte - Reaktive Wartung: Maßnahmen werden erst bei Pannemeldungen eingeleitet - Heterogene Flotte (verschiedene Hersteller, Baujahre) erschwert einheitliches Monitoring
Erwartungen	- Dashboard mit Statusampel für alle Fahrzeuge auf einen Blick - E-Mail-Alert mit konkreter Handlungsempfehlung bei kritischem Zustand - Kosteneinsparungsrechner: vermiedener Ausfall in €
Nutzungskontext	Desktop-Browser (Chrome/Edge), täglich morgens und nach Störungsmeldungen
Rolle im System	Kaufentscheider und primärer Dashboard-Nutzer
Kern-KPIs	Downtime –30 %; Wartungskosten –15 % p.a.; Kosten pro verhinderte Panne: ~600 €/Stunde

Persona 2: Markus Weber – LKW-Fahrer (Endnutzer)	
Alter: 35 Jahre Arbeitgeber: Spedition Müller GmbH Strecke: Fernverkehr, ca. 800 km/Tag, Sattelzug 40 t	
Ziele	Sicher und pünktlich ans Ziel kommen. Liefertermine einhalten, ohne unnötige Stopps oder Pannen auf der Autobahn.
Herausforderungen	- Bemerkt Verschleiß erst, wenn Symptome spürbar werden (z. B. Bremsen ziehen) - Hoher Zeitdruck – Stopps für Kontrollinspektion kosten wertvolle Lieferzeit - Keine proaktive Information über den Fahrzeugzustand vor Fahrtantritt
Erwartungen	- Klare Push-Benachrichtigung in drei Stufen: Info / Warnung / Kritisch - Kurze, verständliche Handlungsempfehlung (keine Rohdaten) - Kein Aufwand – System läuft im Hintergrund

Nutzungskontext	Smartphone (Android/iOS), während oder vor Fahrtantritt
Rolle im System	Empfänger von Push-Alerts; kein Dashboard-Zugang im MVP
Kern-KPIs	Null Pannen auf Autobahn pro Jahr; Warnung ≥ 24 h vor kritischem Ausfall

Persona 3: Stefan Berger – Werkstattleiter (Umsetzer)	
Alter: 52 Jahre Arbeitgeber: Spedition Müller GmbH Team: 8 Mechaniker	
Ziele	Wartungsarbeiten planbar und effizient durchführen. Notfall-Nachtschichten vermeiden. Ersatzteile rechtzeitig bestellen.
Herausforderungen	- Kein Vorlaufsystem – Ausfälle treffen ihn unvorbereitet - Ersatzteile fehlen, wenn die Panne eintritt - Keine datenbasierte Priorisierung: Welches Fahrzeug zuerst in die Werkstatt?
Erwartungen	- Wartungskalender mit vorausberechneten Wartungsfenstern je Fahrzeug - Teilebestellübersicht auf Basis der RUL-Prognose - Priorisierte Liste: Fahrzeuge nach Dringlichkeit sortiert
Nutzungskontext	Desktop-Browser, täglich zu Schichtbeginn
Rolle im System	Empfänger von Wartungsaufträgen; Bestätigung von Wartungsabschluss und Feedback-Geber für den ML-Retraining-Loop
Kern-KPIs	80 % geplante vs. 20 % ungeplante Wartungen; Ersatzteil-Verfügbarkeit ≥ 95 %

3. Value Proposition Canvas

Die folgende Tabelle fasst die zentralen Aufgaben, Schmerzpunkte und Nutzenpotenziale der drei Personas zusammen und verknüpft sie direkt mit den geplanten Systemfunktionen. Die Analyse orientiert sich am Value Proposition Canvas nach Osterwalder & Pigneur (2010).

Kategorie	Thomas – Flottenmanager	Markus – Fahrer	Stefan – Werkstattleiter
Jobs	Flotte verfügbar halten; Wartungskosten senken; ROI nachweisen	Sicher & pünktlich fahren; Liefertermine einhalten	Wartung planbar durchführen; Ersatzteile rechtzeitig bestellen
Pains	Kein Echtzeit-Status; Reaktive Wartung; Keine Vergleichbarkeit	Pannen auf Autobahn; Kein Frühwarnsystem; Zeitdruck	Kein Vorlauf; Ersatzteile fehlen; Ungeplante Notfälle
Gains	ROI durch Ausfallvermeidung; Planbare Budgets; Nachweisbare Einsparungen	Frühwarnung vor kritischen Zuständen; Mehr Sicherheit	Planbare Werkstattzeiten; Optimale Lagerbestände
Pain Relievers	Echtzeit-Dashboard mit Statusampel; Automatische E-Mail-Alerts; Einheitliche Bewertungsskala über alle Fahrzeugtypen	Push-Benachrichtigung in 3 Stufen; Klare Handlungsempfehlung	RUL-basierter Wartungskalender; Teilebestellübersicht
Gain Creators	XGBoost-Klassifikation; Kosteneinsparungsrechner	Info/Warning/Critical Alerts; Warnung ≥ 24 h vor Ausfall; klare Handlungsempfehlung auf dem Handy	RUL-Prognose (XGBoost); Priorisierte Fahrzeugliste; Feedback-Loop: System lernt aus Wartungsergebnissen und wird mit der Zeit präziser

4. Anforderungen

4.1 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben das beobachtbare Verhalten des Systems. Jede Anforderung ist in die drei Felder Beschreibung, Testkriterium und Hintergrund gegliedert.

FA-1: DATENSIMULATION UND DATENERFASSUNG	
Beschreibung	Ein Python-basierter OBD-II-Simulator erzeugt für 5 bis 10 virtuelle LKW einen statischen bzw. chargenweise (Batch) erzeugten Telemetriedatensatz auf Basis des KIT/Kaggle Automotive Engine Health Datensatzes. Simulierte Felder: Motordrehzahl (RPM), Motortemperatur (°C), Öldruck (bar), Bremsflüssigkeitsstand (%), Reifendruck (bar), Kilometerstand, Beladungsgrad (%), Routentyp (Autobahn/Stadtverkehr) und Zeitstempel. Die physikalische Modellierung sorgt dafür, dass höhere Beladung zu schnellerem Komponentenverschleiß führt. Die Ausgabe erfolgt als CSV-Datei; eine Echtzeit-Anbindung (MQTT/Kafka-Streaming mit TimescaleDB) ist als Zielbild vorgesehen, aber kein Bestandteil des MVP.
Testkriterium	Der Simulator läuft für 10 Fahrzeuge über 1.000 simulierte Fahrstunden ohne Fehler durch. Die Ausgabedatei enthält alle definierten Features. Physikalische Korrelationen (Beladung ↔ Verschleiß) sind statistisch nachweisbar. Ein vordefinierter Seed ermöglicht vollständig reproduzierbare Ergebnisse.
Hintergrund	Da im Hochschulprojekt keine echte OBD-II-Hardware zur Verfügung steht, ist der Simulator die primäre Datenquelle. Die Architektur ist so gestaltet, dass er im Zielbild durch einen echten MQTT/Kafka-Stream ersetzt werden kann, ohne die nachgelagerte ML-Pipeline zu verändern.

FA-2: DATENANREICHERUNG ÜBER EXTERNE APIS	
Beschreibung	Das System ruft ergänzende Umgebungsdaten über zwei externe REST-APIs ab. Die OpenWeatherMap API (Free Tier: 1.000 Calls/Tag) liefert Temperatur und Niederschlag als zusätzliche Verschleißfaktoren. Die CarAPI stellt eine Datenbank mit über 9.400 OBD-II-Fehlercodes (DTCs) bereit, die für realistische Alarmmeldungen im Dashboard verwendet werden. Beide Datenquellen werden mit den Simulator-Daten über Fahrzeug-ID und Zeitstempel verknüpft.
Testkriterium	API-Aufrufe an OpenWeatherMap und CarAPI werden erfolgreich ausgeführt. Die zurückgegebenen Wetterdaten werden korrekt an die zugehörigen Fahrtdaten gejoined. Für mindestens drei unterschiedliche DTCs werden korrekte Fehlerbeschreibungen im Dashboard angezeigt. Bei API-Ausfall wird ein definierter Fallback-Wert verwendet.
Hintergrund	Externe Signale erhöhen die Modellqualität (Wetter als Korrelationsvariable für Bremsverschleiß im Winter) und steigern die Realitätsnähe der Demo-Szenarien erheblich.

FA-3: ANOMALIEERKENNUNG (ISOLATION FOREST)	
Beschreibung	Ein Isolation-Forest-Modell (scikit-learn, Unsupervised) erkennt anomale Sensormuster in Bremsen- und Motordaten. Das Modell wird auf dem normalisierten Trainingsdatensatz trainiert und liefert für jeden eingehenden Datenpunkt einen Anomalie-Score zwischen -1 (Anomalie) und +1 (normal).

	Anomalien ab einem konfigurierbaren Schwellenwert werden als INFO-Alert weitergeleitet und in TimescaleDB protokolliert.
Testkriterium	Ein synthetisches Degradationsszenario (LKW-01: linearer Rückgang der Bremsflüssigkeit von 80 % auf 8 % über 500 Fahrstunden) wird korrekt als Anomalie erkannt, bevor der Wert die 15%-Warnschwelle unterschreitet. Die False-Positive-Rate im Normalbetrieb liegt unter 10 %.
Hintergrund	Der Isolation Forest benötigt keine gelabelten Ausfallhistorien und eignet sich daher besonders gut als erster Erkennungsschritt für neue Fahrzeugtypen oder bisher unbekannte Fehlermuster.

FA-4: VERSCHLEIßKLASSIFIKATION UND RUL-PROGNOSE (XGBOOST)

Beschreibung	Ein XGBoost-Klassifikationsmodell (Supervised) stuft den aktuellen Fahrzeugzustand in drei Kategorien ein: OK, Warnung und Kritisch. Als Trainingsgrundlage dient der KIT/Kaggle Automotive Engine Health Datensatz (ca. 1.000 Fahrzeuge, Labels: Excellent / Good / Needs Service / Very Bad). Zusätzlich schätzt ein XGBoost-Regressionsmodell die verbleibende Restlebensdauer (Remaining Useful Life, RUL) in Kilometern oder Betriebsstunden bis zur notwendigen Wartung. Beide Modellausgaben werden mit jedem neuen Datenbatch aktualisiert.
Testkriterium	Das Klassifikationsmodell erreicht auf dem Hold-out-Testset eine Gesamtgenauigkeit von mindestens 80 %. Die Klasse Kritisch erzielt einen Recall von mindestens 85 % (Sicherheitspriorität: kein Ausfall darf unerkannt bleiben). Die RUL-Prognose weicht im Mittel um weniger als 15 % vom tatsächlichen Simulationswert ab.
Hintergrund	Die Kombination aus Klassifikation und RUL-Prognose bildet den zentralen Mehrwert des Systems. Sie erlaubt es Flottenmanagern, Wartungsfenster präzise zu planen und die tatsächlich eingesparten Kosten zu beziffern.

FA-5: ALERT-ENGINE (DREISTUFIGES WARNSYSTEM)

Beschreibung	Eine regelbasierte Alert-Engine wertet die Ausgaben der ML-Modelle aus und erzeugt strukturierte Warnmeldungen in drei Schweregrad-Stufen: INFO (Anomalie erkannt, kein sofortiger Handlungsbedarf), WARNUNG (Verschleiß erhöht, Wartung innerhalb von 14 Tagen empfohlen) und KRITISCH (sofortige Maßnahme erforderlich, Fahrt ggf. unterbrechen). Jeder Alert enthält: Fahrzeug-ID, Zeitstempel, betroffene Komponente, Schweregrad, Handlungsempfehlung und geschätzte Kosteneinsparung (bei vermiedenem Ausfall: ~600 €). Kritische Alerts werden zusätzlich per E-Mail (SMTP-Mock) versandt.
Testkriterium	Für alle drei Schweregrade werden korrekte Alerts ausgelöst. Die vollständige Batch-Verarbeitung von 10 Fahrzeugen – von der CSV-Einlesung über ML-Inferenz bis zur Anzeige im Dashboard – dauert weniger als 30 Sekunden. Der E-Mail-Versand (SMTP-Mock) wird erfolgreich protokolliert. Eine fehlerhafte Eingabe (z. B. fehlender Sensor-Wert) löst eine definierte Fehlerbehandlung aus, ohne das System zum Absturz zu bringen.
Hintergrund	Die Alert-Engine ist der zentrale Aha-Moment der Demo: Sensordegradation wird sichtbar, das ML-Modell reagiert, und der Nutzen (vermiedener Ausfall = Geld gespart) wird unmittelbar spürbar.

FA-6: STREAMLIT-DASHBOARD

Beschreibung	Ein webbasiertes Dashboard (Python Streamlit) stellt die Systemausgaben visualisiert dar. Es besteht aus drei Hauptansichten: (1) Flottenübersicht – Statusampel (OK/Warnung/Kritisch) für alle Fahrzeuge, Zusammenfassung der Kennzahlen. (2) Einzelfahrzeug-Detail – Sensor-Zeitreihen (Liniendiagramme), aktuelle Verschleißklasse, RUL-Balkendiagramm, Alert-Verlauf je Fahrzeug. (3) Alert-Feed – chronologische Liste aller Warnungen mit Filterfunktion nach Schweregrad. Darüber hinaus steht ein CSV-Export der Analyseergebnisse zur Verfügung.
Testkriterium	Die Flottenübersicht lädt mit 10 Fahrzeugen in unter 3 Sekunden. Der Drilldown auf ein Einzelfahrzeug funktioniert korrekt. Der Alert-Feed zeigt alle Stufen in der richtigen Reihenfolge an. Der CSV-Export enthält alle relevanten Felder. Das Dashboard ist im Chrome-Browser ohne Fehlermeldungen nutzbar.
Hintergrund	Streamlit ermöglicht schnelles Prototyping ohne JavaScript-Kenntnisse und ist als Python-native Lösung optimal in die ML-Pipeline integrierbar. Im Zielbild wird Streamlit durch eine produktionsreife Webapplikation (z. B. React/Angular + FastAPI) ersetzt.

FA-7: DEMO-SZENARIO BREMSPAD-DEGRADATION (LKW-01)

Beschreibung	Ein vordefiniertes Demo-Szenario simuliert die vollständige Systemkette: LKW-01 zeigt einen schrittweisen Rückgang der Bremsflüssigkeit von 80 % auf 8 % über ca. 500 Fahrstunden. Der Isolation Forest erkennt ab ca. 60 % eine Anomalie (INFO). Der XGBoost-Klassifikator stuft das Fahrzeug bei ca. 25 % als Warnung und bei ca. 12 % als Kritisch ein. Die Alert-Engine löst E-Mail und Dashboard-Benachrichtigung aus. Der Kosteneinsparungsrechner zeigt: 1 verhinderte Panne = ~600 € vermiedene Kosten.
Testkriterium	Das Demo-Szenario läuft vollständig durch und alle Pipeline-Stufen (Simulation → ML → Alert → Dashboard) sind sichtbar aktiv. Die Gesamtdurchlaufzeit der Demo-Pipeline beträgt weniger als 2 Minuten in Echtzeit.
Hintergrund	Das Demo-Szenario ist das zentrale Präsentationsstück für die Abschlusspräsentation am 10.07. Es macht das Kernversprechen des Systems – Sensordegradation sichtbar machen, bevor der Schaden eintritt – erlebbar.

FA-8: FEEDBACK-LOOP UND NÄCHTLICHES RETRAINING

Beschreibung	Das System erfasst nach jeder Wartung strukturiertes Feedback des Werkstattleiters zum Wartungsausgang: bestätigt (Warnung war korrekt) oder Fehlalarm (Warnung war nicht gerechtfertigt). Jedes Feedback wird als neues Trainingslabel mit Fahrzeug-ID, Zeitstempel, betroffener Komponente und Ausgangskategorie persistiert. Nächtlich (täglich um 02:00 Uhr) startet ein automatisierter Retraining-Job, der die XGBoost-Modelle (Klassifikation und RUL) auf dem erweiterten Datensatz neu trainiert. Das neue Modell wird erst aktiviert, wenn es auf dem Hold-out-Testset die bisherigen Accuracy- und Recall-Metriken mindestens hält (automatische Qualitätssicherung).
Testkriterium	Ein im Dashboard abgegebenes Feedback (bestätigt/Fehlalarm) wird korrekt in der TimescaleDB persistiert und erscheint im nächsten Retraining-Datensatz. Der nächtliche Retraining-Job läuft vollständig durch und schließt in weniger als 2 Stunden ab. Ein Modell mit schlechterer Performance wird automatisch verworfen und das Vorgängermodell bleibt aktiv. Ein Qualitätsreport (Accuracy, Recall, MAE vorher/nachher) wird protokolliert.

Hintergrund	Der Feedback-Loop ist die zentrale Differenzierung des Systems gegenüber einer einmaligen ML-Analyse. Er sorgt dafür, dass das System flottenindividuell lernt, sich an neue Fahrzeugtypen anpasst und mit der Zeit präziser wird. Die automatische Qualitätsprüfung verhindert, dass schlechte Feedback-Daten die Modellgüte verschlechtern. Damit ist aus einem einmaligen Analyseprojekt ein produktiver Service geworden, der mit jeder Wartung besser wird.
--------------------	--

4.2 Nichtfunktionale Anforderungen

Kategorie	Anforderung
Latenz	Die vollständige Batch-Pipeline (CSV-Einlesung → ML-Verarbeitung → Alert-Generierung → Dashboard-Aktualisierung) für 10 Fahrzeuge dauert weniger als 30 Sekunden. Das Dashboard reagiert auf Nutzerinteraktionen (z. B. Drilldown auf ein Fahrzeug) innerhalb von 3 Sekunden. Echtzeit-Latenz ist kein Ziel des MVP – die Daten werden chargenweise verarbeitet, nicht als Live-Stream.
Simulationskapazität	Das System verarbeitet Batch-Daten von bis zu 100 simulierten Fahrzeugen ohne Performance-Einbruch. Der vollständige ML-Batch-Durchlauf (Anomalieerkennung + Klassifikation + RUL) für 100 Fahrzeuge dauert weniger als 30 Sekunden.
Reproduzierbarkeit	Der Python-Simulator verwendet einen konfigurierbaren zufälligen Seed (Standard: 42). Alle Demo-Szenarien liefern damit bei gleichen Eingaben identische Ergebnisse und sind jederzeit reproduzierbar.
Datenschutz (DSGVO)	Es werden keine echten Personendaten verarbeitet. Fahrzeug-IDs sind pseudonymisiert (LKW-01 bis LKW-XX). Alle Daten verbleiben lokal (kein Cloud-Upload im MVP). Externe API-Aufrufe übermitteln ausschließlich allgemeine Standort- und Zeitinformationen.
Wartbarkeit	Der Code ist modular aufgebaut: Simulator, ML-Pipeline, Alert-Engine und Dashboard sind voneinander unabhängige Python-Module mit definierten Schnittstellen. Eine README-Datei dokumentiert Setup und Ausführung. Alle Abhängigkeiten sind in einer requirements.txt festgelegt.
Verfügbarkeit (MVP)	Das System wird lokal ausgeführt (kein Cloud-Deployment im MVP). Die Streamlit-Anwendung startet lokal innerhalb von 30 Sekunden nach dem Aufruf.
Datenpersistenz	Simulierte Rohdaten und ML-Ergebnisse werden in einer lokalen TimescaleDB-Instanz gespeichert. Der Dashboard-Zustand (Alert-History, Sensor-Zeitreihen) bleibt über Neustarts der Anwendung erhalten.
Skalierbarkeit (Zielbild)	Die Systemarchitektur ist so konzipiert, dass CSV-Batch-Verarbeitung durch echtes MQTT/Kafka-Streaming ersetzt werden kann, ohne die ML-Pipeline umzuschreiben. Streamlit kann durch eine produktionsreife Webapplikation ergänzt oder ersetzt werden.
Testbarkeit	Jede funktionale Anforderung (FA-1 bis FA-8) enthält ein konkretes, manuell überprüfbares Testkriterium. Für die ML-Modelle werden quantitative Metriken (Accuracy, Recall, MAE) als Akzeptanzkriterium definiert.
Zugriffsschutz	Der Dashboard-Zugriff ist durch eine rollenbasierte Login-Authentifizierung abgesichert. Flottenmanager und Werkstattleiter erhalten unterschiedliche Sichtrechte. API-Keys werden ausschließlich über Umgebungsvariablen

	(.env) verwaltet; es dürfen keine Secrets im Code oder Repository gespeichert sein.
Deployment	Das gesamte System ist über Docker Compose mit einem einzigen Befehl (docker compose up) auf jedem Docker-fähigen System reproduzierbar deploybar. Eine README-Datei dokumentiert alle Voraussetzungen und den Setup-Prozess vollständig.

5. Anwendungsfall

5.1 Aktivitätsdiagramm

Das folgende Aktivitätsdiagramm beschreibt die typische Nutzung des Systems aus Sicht von Thomas Müller (Flottenmanager). Der Prozessfluss gliedert sich in vier logische Phasen: Authentifizierung, Datenladung, Analyse und Reaktion.

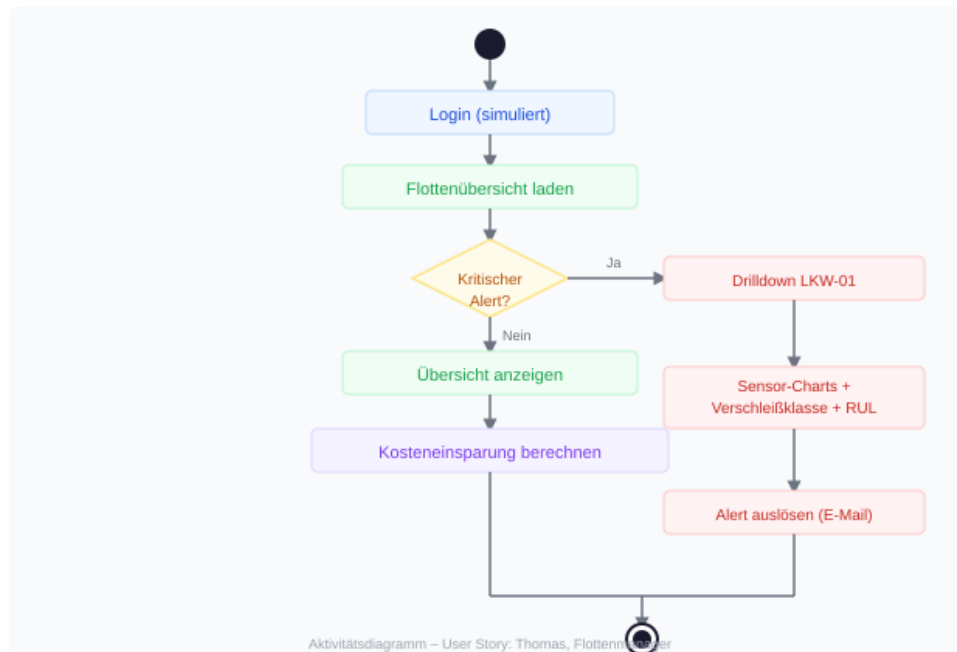


Abbildung 1: Aktivitätsdiagramm – User Story: Thomas, Flottenmanager

Nach dem Login lädt das System die aktuelle Flottenübersicht. Liegt ein kritischer Alert vor, kann der Nutzer direkt in die Einzelfahrzeug-Detailansicht verzweigen, wo Sensor-Zeitreihen, Verschleißklasse und RUL-Prognose einsehbar sind. Die Alert-Engine löst parallel E-Mail-Benachrichtigungen aus. Abschließend berechnet der Kosteneinsparungsrechner den finanziellen Mehrwert der verhinderten Panne.

5.2 UI-Wireframes

Die folgenden Wireframes zeigen die drei zentralen Bildschirmansichten des Streamlit-Dashboards. Sie dienen als visuelle Entwurfsgrundlage und beschreiben Informationsarchitektur, Anordnung der Komponenten und das dreistufige Alert-Farbsystem (Grün / Orange / Rot).

Screen 1: Flottenübersicht

Die Übersicht zeigt alle Fahrzeuge in einer Tabelle mit farbcodierten Statusindikatoren. Die vier Kennzahlenkarten oben ermöglichen eine sofortige Lageeinschätzung. Kritische Fahrzeuge werden rot hervorgehoben und erscheinen in der Tabellenansicht ganz oben.

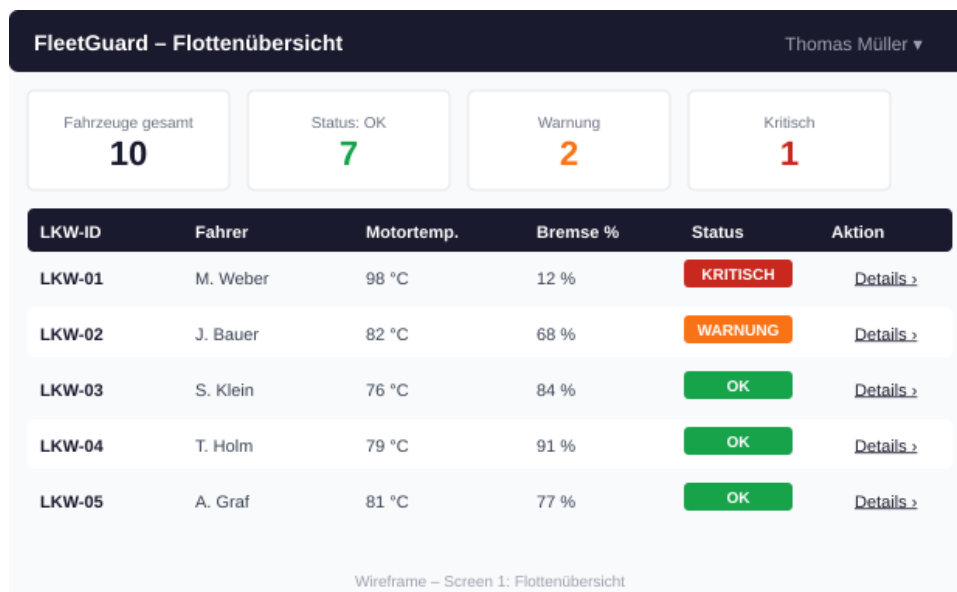


Abbildung 2: Wireframe – Screen 1: Flottenübersicht

Screen 2: Einzelfahrzeug-Detail

Die Detailansicht zeigt alle Sensormesswerte des ausgewählten Fahrzeugs als Balkenindikatoren (links) und die historische Entwicklung des kritischen Sensors als Zeitreihe (rechts). Der Alert-Verlauf am unteren Rand dokumentiert alle bisherigen Warnungen chronologisch.

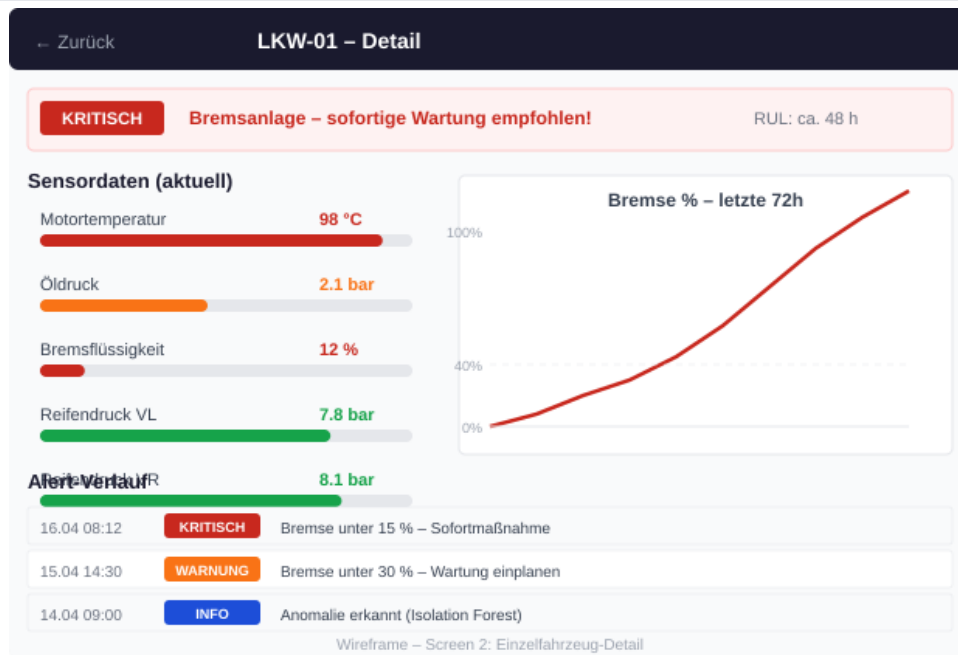


Abbildung 3: Wireframe – Screen 2: Einzelfahrzeug-Detail (LKW-01)

Screen 3: Alert-Feed

Der Alert-Feed listet alle Systemwarnungen chronologisch auf. Eine Filterleiste ermöglicht die Einschränkung auf eine bestimmte Schweregrad-Stufe. Jeder Alert zeigt Zeitstempel, Stufe, betroffenes Fahrzeug, Meldungstext und einen direkten Link zur Detailansicht.

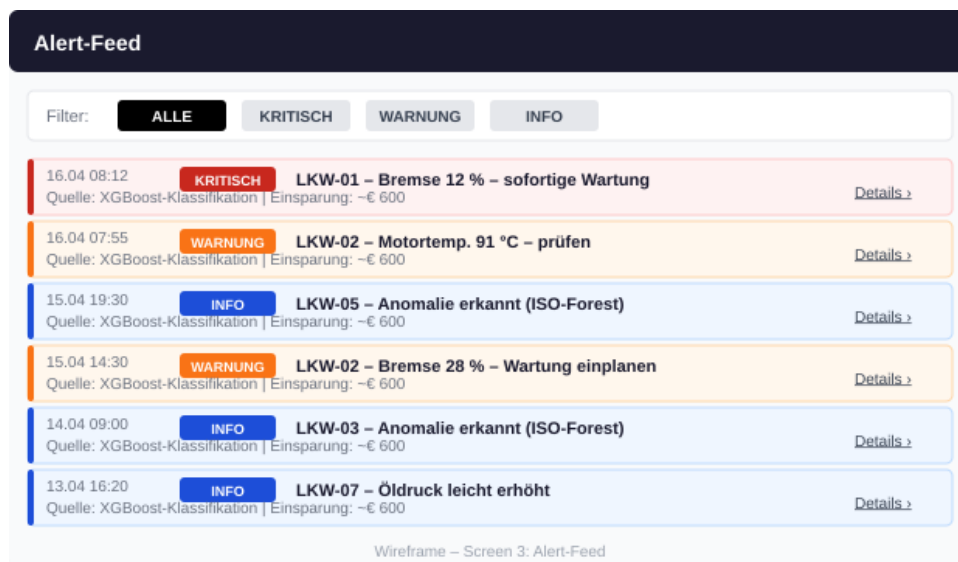


Abbildung 4: Wireframe – Screen 3: Alert-Feed

6. Architektur

6.1 Systemarchitektur – 5-Schichten-Modell

Die Systemarchitektur folgt einem klar abgegrenzten 5-Schichten-Modell, das sowohl den MVP-Scope als auch die Zielbild-Erweiterungen berücksichtigt. Die Schichttrennung stellt sicher, dass einzelne Komponenten (z. B. der CSV-Datentransport im MVP) im Zielbild durch produktionsreife Alternativen (Kafka/MQTT) ersetzt werden können, ohne andere Schichten zu beeinflussen.

Systemarchitektur & Datenfluss

Fünf-Schichten-Modell · MVP vs. Zielbild



Abbildung 5: 5-Schichten-Systemarchitektur mit MVP-Scope-Abgrenzung

6.2 Datenfluss und Machine Learning Pipeline

Der Datenfluss im MVP verläuft sequenziell durch alle fünf Schichten:

- Schicht 1 – Datengenerierung: Der Python-Simulator erzeugt OBD-II-Telemetriedaten für 5–10 LKW als CSV-Datei. Der KIT/Kaggle-Datensatz liefert reale OBD-II-Referenzwerte als Trainingsgrundlage.
- Schicht 2 – Datentransport (MVP: Batch-CSV): Die CSV-Datei wird per Pandas-DataFrame eingelesen und als simulierter Datenstrom an Schicht 3 weitergegeben. Im Zielbild ersetzt ein MQTT-Broker + Apache-Kafka-Cluster diesen Schritt.
- Schicht 3 – Datenanreicherung und Storage: Wetterdaten (OpenWeatherMap) und DTC-Codes (CarAPI) werden per REST-API abgefragt und dem Datensatz hinzugefügt. Die angereicherten Daten werden in einer lokalen TimescaleDB-Instanz gespeichert (PostgreSQL mit Timescale-Extension; Hypertables + Continuous Aggregates für zeitreihen-optimierte Queries).
- Schicht 4 – ML-Verarbeitung: Isolation Forest erkennt Anomalien (unsupervised). XGBoost klassifiziert den Zustand (OK/Warnung/Kritisch); Random Forest schätzt die Remaining Useful Life (RUL) als Regressionsaufgabe. Die Alert-Engine setzt ML-Ausgaben in strukturierte Warnmeldungen um.
- Schicht 5 – Ausgabe: Das Streamlit-Dashboard stellt alle Ergebnisse visualisiert bereit. Kritische Alerts werden zusätzlich per SMTP (Mock) als E-Mail versandt.

ML-Pipeline – vom Rohdatum zur Wartungsempfehlung

Training offline · Inference batch · Feedback-Loop schliesst den Kreis

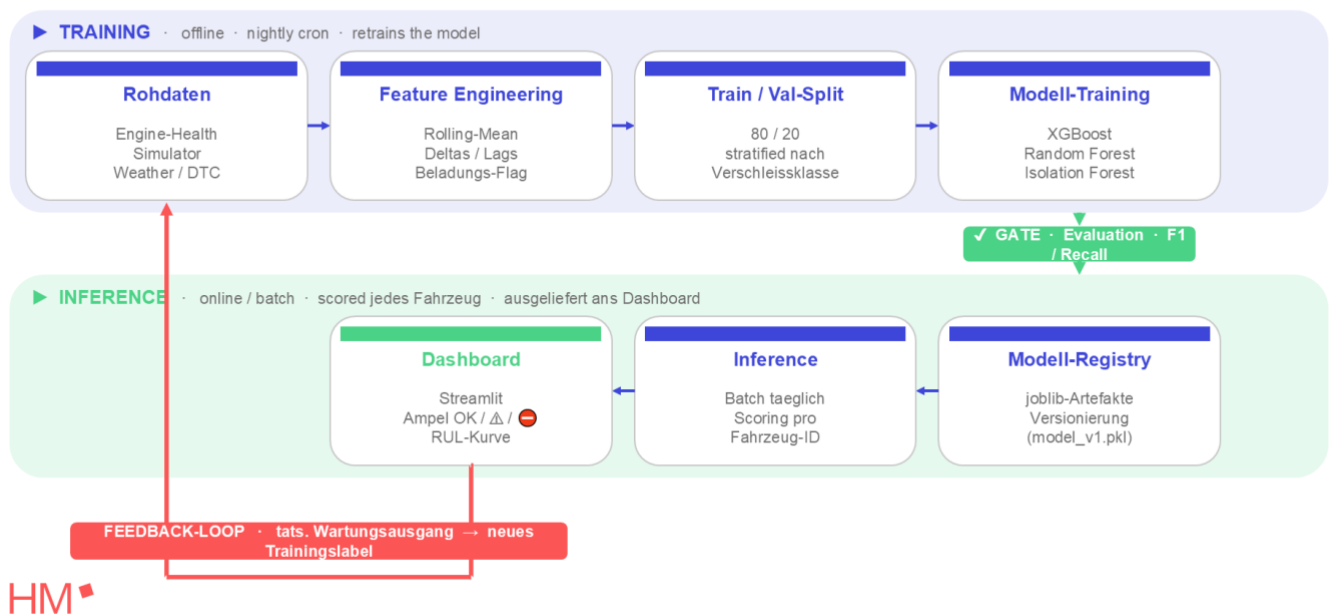


Abbildung 6: ML-Pipeline mit Training, Inference und Feedback-Loop

Die ML-Pipeline operationalisiert Schicht 4 des Datenflusses (vgl. Abschnitt 6.2) und trennt konsequent zwischen Training und Inference. Sie besteht aus vier aufeinander aufbauenden Trainingsstufen, einem Qualitäts-Gate zur Modell-Freigabe, drei Inference-Stufen zur Auslieferung an das Dashboard sowie einem Feedback-Loop, der tatsächliche Wartungsausgänge als neue Trainingslabels zurückführt.

Trainingsphase

Der Trainingszweig wird einmal pro Nacht automatisiert ausgeführt und läuft in vier Schritten ab. Aus den Rohdaten (Engine-Health-Datensatz, Python-Simulator und APIs für Wetter und DTC-Fehlercodes) werden im Feature-Engineering aussagekräftige Kennzahlen abgeleitet (Rolling Means zur Glättung, Deltas und Lags zur Erkennung von Zeitverläufen, Beladungs-Flag als Kontextmerkmal). Der stratifizierte 80/20-Split stellt sicher, dass seltene Verschleißklassen in Trainings- und Validierungsdaten gleichermaßen vertreten sind. Das anschließende Modell-Training trainiert drei Algorithmen parallel: Isolation Forest erkennt Anomalien ohne Label-Anforderung, XGBoost klassifiziert den Zustand in die drei Stufen OK, Warnung und Kritisch, und Random Forest schätzt die Remaining Useful Life (RUL) als Regressionsaufgabe.

Gate (Evaluation)

Vor der Übernahme in den Produktivbetrieb passiert jedes neu trainierte Modell ein Qualitäts-Gate. Bewertet werden F1-Score, Recall und Confusion-Matrix auf dem Validierungsset; für Predictive Maintenance ist der Recall die wichtigste Metrik, da ein verpasster kritischer Fahrzeugzustand kostspieliger ist als ein Fehlalarm. Nur Modelle, die definierte Mindestschwellen erreichen, werden in die Modell-Registry übernommen. Ältere Modellversionen bleiben als Fallback verfügbar, sodass bei Verschlechterung der Metriken ein Rollback auf eine früher freigegebene Version möglich ist.

Inference-Phase

Im produktiven Einsatz wird die jeweils aktuelle Modellversion aus der Registry (versionierte joblib-Artefakte, z. B. model_v1.pkl) geladen und einmal täglich im Batch-Modus auf neu eingetroffene Sensordaten angewendet. Jedes Fahrzeug der Flotte erhält einen Klassifikations-Score und eine RUL-Prognose. Das Streamlit-Dashboard stellt die Ergebnisse als dreistufige Ampel (OK, Warnung, Kritisch) dar und ergänzt eine

RUL-Kurve je Komponente. Kritische Alerts werden zusätzlich per SMTP-Mock als E-Mail an den Flottenmanager versandt.

Feedback-Loop

Die rote Rückkopplungsschleife schließt den Kreis. Sobald ein vom Werkstattleiter bestätigter Wartungsausgang vorliegt (tatsächlicher Defekt oder Fehlalarm), wird dieses Ergebnis als neues Trainingslabel in die Rohdaten zurückgeführt. Mit dem nächsten nächtlichen Training fließen die kumulierten Feedback-Labels in den Trainingsdatensatz ein und verbessern das Modell kontinuierlich. Diese Rückkopplung ist der zentrale Unterschied zu einer einmaligen ML-Analyse: Sie macht das System zu einem sich selbst verbessernden Predictive-Maintenance-Service, dessen Genauigkeit im Zeitverlauf steigt.

6.3 Technologie-Stack

Schicht	Technologie	Begründung
Datensimulation	Python 3.11, NumPy, Pandas	Leichtgewichtig, physikalisch modellierbar, kein Infrastruktur-Overhead
Datentransport	Batch-CSV (MVP) / Kafka (Zielbild)	Kein Broker-Setup im MVP; Architektur erlaubt späteren Wechsel ohne ML-Umbau
Storage	PostgreSQL mit TimescaleDB	Optimiert für Zeitreihen; SQL-basiert; nahtlos in Python integrierbar
ML – Anomalie	scikit-learn: Isolation Forest	Kein gelabeltes Training nötig; robust bei unbekannten Fehlermustern
ML – Klassifikation	XGBoost	State-of-the-Art für tabellarische Daten; schnelles Training, hohe Genauigkeit
ML – RUL	XGBoost Regression	Gleiche Pipeline wie Klassifikation; LSTM als Zielbild für Zeitreihen-RUL
API-Anreicherung	OpenWeatherMap REST, CarAPI REST	Beide kostenlos im Free Tier; JSON-Rückgabe, einfache Integration
Frontend	Streamlit (Python)	Python-nativ, kein JavaScript erforderlich; schnelles Prototyping
Alert-Versand	Python smtplib (SMTP-Mock)	Einfach testbar ohne echten Mail-Server; produktionsreif austauschbar
Entwicklungsumgebung	Python venv, GitHub, Jupyter Notebook	Standardwerkzeuge, kurskompatibel
Deployment	Docker Compose	Einheitliche Umgebung; Einfache Weitergabe des Projekts

6.4 Datenquellen

Datensatz / API	Typ	Zugang	Verwendung im MVP	Status
KIT/Kaggle Engine Health	CSV Download	Kostenlos	ML-Training (Labels: Excellent/Good/Needs Service/Very Bad)	MVP-ready
Python-OBD-II-Simulator	Eigenentwicklung	Kostenlos	Telemetriedaten (5–10 LKW, alle Features)	MVP-ready
OpenWeatherMap API	REST API / JSON	Free Tier (1.000 Calls/Tag)	Wetter als Verschleiß-Feature	MVP-ready
CarAPI (DTCs)	REST API / JSON	Free Developer Tier	9.400+ Fehlercodes für Dashboard-Alerts	MVP-ready
CWRU Bearing Dataset	CSV Download	Kostenlos	Radlager-Überwachungsmodul (separates Modul)	Zielbild

7. Projektabgrenzung und Ausblick

7.1 Was das System bewusst nicht leisten wird (MVP)

Die folgende Tabelle gibt eine vollständige Übersicht der bewussten Ausgrenzungen des MVP und deren Begründung:

Kategorie	Anforderung
Echte Hardware	Kein physischer OBD-II-Dongle / ELM327. Alle Daten stammen aus dem Python-Simulator.
Echtzeit-Streaming	Kein produktiver Kafka-Broker oder MQTT-Stack. CSV-Batch-Verarbeitung ersetzt den Stream.
ERP / SAP-Integration	Keine Anbindung an Dispositionssysteme oder Warenwirtschaft. Wartungsaufträge werden manuell angelegt.
Mobile App	Kein natives App für iOS oder Android. Fahrer-Alerts erfolgen im MVP per E-Mail.
LSTM / Deep Learning	LSTM und Autoencoder sind im Zielbild vorgesehen; im MVP fokussiert das Projekt auf Isolation Forest + XGBoost.
Cloud-Deployment	Kein AWS / Azure / GCP im MVP. Alle Komponenten laufen lokal. Cloud-Deployment ist als Zielbild definiert.
DSGVO-Compliance	Nur simulierte Pseudodaten. Vollständige DSGVO-Prüfung erst für Produktivbetrieb erforderlich.

7.2 Zielbild nach dem MVP

Nach erfolgreichem Abschluss des MVP sind folgende Erweiterungen für den produktiven Einsatz vorgesehen:

- Echtes MQTT/Kafka-Streaming mit ELM327-Dongles oder CAN-Bus-Schnittstellen
- LSTM-basierte RUL-Prognose für langfristige Zeitreihenvorhersage
- Cloud-Deployment (AWS oder Azure) mit automatischem Skalieren
- Native Mobile App für Fahrer (React Native oder Flutter)
- SAP/ERP-Integration für automatische Werkstattauftragsanlage
- Zertifizierte DSGVO-Konformität und BSI-IT-Grundschutz

– Ende des Pflichtenhefts –